<!-- SPDX-License-Identifier: GPL-3.0-or-later Copyright (c) 2026 Mohamed Hammad -->

Loran — Specification v0.2 (Draft)

Field	Value
Project	Loran
Tagline	The Steelbore reference manual.
Version	0.2.0 (specification draft)
Date	2026-05-11
Author	Mohamed Hammad
Maintainer	Mohamed Hammad Mohamed.Hammad@Steelbore.com
Copyright	(c) 2026 Mohamed Hammad
License	GPL-3.0-or-later
Website	https://Loran.Steelbore.com/
Governed by	Steelbore Standard v1.1, Steelbore SFRS v1.0.0

Changes since v0.1 (2026-04-29):

- **Renamed** Lodestone → Loran. The compass/navigation metaphor is preserved and sharpened: LORAN (LOng RAnge Navigation) was the radio navigation infrastructure used by ships and aircraft from the 1940s until GPS retired it in 2010 — precision wayfinding for the era before satellite positioning.
- **Renamed** Lattice → Bravais throughout (project name, overlay paths, `/etc/os-release` ID, distro overlay codename in examples and phasing). Bravais refers to the 14 unique Bravais lattices in crystallography — a sharper crystallographic identity than the generic "Lattice."
- **Split** the `show` verb into `show` (curated-or-fail) and `help` (always-live `--help` capture). Brand boundary preserved; resolution chain simplified.
- **Renamed** frontmatter field `language` → `written_in`. The `language` identifier is reserved for future i18n use.
- **Added** frontmatter field `safe_alias_for` — a strict subset of `replaces` flagging which legacy tools the modern entry can safely be aliased to without breaking common-case scripts.
- **Added** frontmatter field `pairs_with` — non-reciprocal recommendation set for companion tools.

- **Promoted** page signing from open question to a Phase 2 (Billet) requirement: minisign + ed25519 + `minisign-verify` crate.
- **Added** exit code `TARBALL_VERIFY_FAILED = 11`.
- **Added** `Ingestor` trait abstraction in §3 to enable Phase 3 ingestion of SFRS `describe` output from other Steelbore CLIs.
- **Clarified** that `category` is slash-tolerant for future nested-hierarchy UX.
- **Added** `NOTICE.md` to required posture files per Steelbore Standard v1.1 §5.2.
- **Resolved** three former open questions: per-distro overlay source-of-truth (per-distro repos, surfaced via tarball pipeline); i18n directory layout (tldr-pages `pages.<lang>/` precedent); nested-category UX threshold (~50 entries).

---

1. Purpose

Loran is the canonical, agent-friendly reference tool for Steelbore-based systems (Bravais, Ferrite OS, future distros). It answers three questions in one binary:

1. **What tools are available on this system?** — categorised browse of the Steelbore tool catalog.
2. **What does this tool do, and what does it replace?** — Steelbore-curated intro, then tldr page as fallback.
3. **What replaces the legacy tool I know?** — `loran find ls` → `eza`.

Loran is to Steelbore what `man` is to Unix and `info` is to GNU: a system-level reference. Unlike either, it ships agent-native (`--json`, `schema`, MCP) from day one.

The name: LORAN (LOng RAnge Navigation) was the radio navigation system used by ships and aircraft from the 1940s until 2010, when GPS finally retired it. It was the precision wayfinding infrastructure of its era — the reference grid you trusted when you needed to know exactly where you were. Loran the tool is the precision reference grid for a Steelbore system: the catalog of curated tool knowledge you trust when you need to know what's available and what to use.

---

2. Locked Design Decisions

These are settled and inform the rest of this spec. Listed up front so reviewers can challenge the foundations before spending time on details.

#	Decision
1	**Name:** Loran (heritage engineering acronym — LORAN, radio navigation system; navigational metaphor)
2	**Greenfield Cargo workspace.** No fork of tealdeer or tlrc. Reuse patterns, not code.
3	**Global registry + overlays** for the page collection (upstream / per-distro / per-user).
4	**Category-first browsing** (mirrors the GRUB/parted help affordance).
5	**TUI default with TTY auto-detection** per SFRS §5 cascade. Pipe = `--format json`.
6	**Tarball update model**, mirroring tldr-pages. No git client dependency at runtime. Tarballs are minisign-signed.
7	**Verb split:** `loran show <tool>` is curated-or-fail (no live fallback); `loran help <tool>` is always-live `--help` capture.
8	**Resolution order for `show`:** Steelbore intro (always) → custom page → tldr page → no-entry prompt. **Live `--help` is never invoked by `show`.**
9	**Page format:** single Markdown file with TOML frontmatter (Hugo/Zola style).
10	**In-process rendering** of Loran's curated pages via `pulldown-cmark` + `ratatui` + `crossterm`. No `bat` for own pages.
11	**De-themed rendering** for `loran help` captures: monochrome frame, NOT the Steelbore palette. Brand cues reserved for curated content.
12	`bat -pp` as PAGER/MANPAGER for the `loran help` subprocess only. Falls back to `cat` if `bat` absent.
13	`loran new <tool>` scaffolds pages from a user-editable template, writing to the user overlay by default.
14	`replaces` (broad set) + `safe_alias_for` (strict subset, alias-safe). Validated at index build: `safe_alias_for ⊆ replaces`.
15	`written_in` in frontmatter (implementation language). `language` is reserved for future i18n.
16	`category` is slash-tolerant: `system/file-listing` is valid today; flat UX in v1, nested UX deferred.
17	**MCP surface is read-only.** Agents can `list`, `show`, `find`, `search`, `categories`. No `update`, `new`, or `validate` via MCP.

#	Decision
18	**GPL-3.0-or-later**, SPDX headers on all source files (Steelbore Standard §4). Posture files (README, NOTICE, CONTRIBUTING, LICENSE) per Standard v1.1 §5.2.

3. Architecture

3.1 Cargo Workspace Layout

```
loran/
├── Cargo.toml                  # workspace root
├── README.md                   # includes Project Posture section
├── NOTICE.md                   # no-warranty / no-liability statement
├── CONTRIBUTING.md             # human onboarding, sign-off, scope
├── LICENSE                     # GPL-3.0-or-later verbatim
├── AGENTS.md                   # generic agent context
├── CLAUDE.md                   # Claude Code-specific context
├── SKILL.md                    # capability surface for Steelbore Skills
├── crates/
│   ├── loran-cli/              # clap binary, dispatcher, exit codes
│   ├── loran-core/             # orchestration, resolution chains
│   ├── loran-index/            # index builder + Ingestor trait
│   ├── loran-pages/            # page parser (TOML frontmatter + body)
│   ├── loran-render/           # Markdown → terminal renderer
│   ├── loran-tldr/             # tldr tarball fetch + cache + lookup
│   ├── loran-tui/              # ratatui app (browse, detail, search)
│   └── loran-mcp/              # MCP server surface (Phase 3, read-only)
├── pages/                      # bundled fallback pages (built into binary)
│   └── …
└── xtask/                      # build/release/index-validate helpers
```

Crate-prefix naming follows mainstream Rust workspace convention. Project-level metallurgical naming applies at the Steelbore-Standard layer (see §13); internal crates are implementation detail. Release codenames follow the Standard's cast-form list (Ingot → Billet → Bloom).

3.2 The `Ingestor` Trait

`loran-index` exposes an `Ingestor` trait so the index loader is pluggable rather than hard-coded to a single source format. v1 ships one implementation: `MarkdownPagesIngestor` (TOML frontmatter + Markdown body). Phase 3 adds `DescribeIngestor`, which invokes `<tool> describe --json` against SFRS-compliant Steelbore binaries on `$PATH` and synthesises baseline entries — making the Steelbore ecosystem self-documenting (every new Steelbore CLI gets a Loran entry for free, with curated pages overlaying on top where they exist).

This abstraction is non-load-bearing in v1 but designed-in from Phase 1 so retrofitting it later doesn't require restructuring the index pipeline.

3.3 Dependency Stack (canonical picks)

Concern	Crate	Rationale
CLI parsing	`clap` (derive)	SFRS canonical
Serialization	`serde`, `serde_json`, `toml`	SFRS canonical
Markdown parsing	`pulldown-cmark`	CommonMark, fast, no_std-friendly
TUI	`ratatui` + `crossterm`	SFRS canonical
Time	`jiff`	Steelbore Standard §12.5 preferred
HTTP (tarball)	`ureq` + `rustls`	Lean, no async runtime needed for one-shot fetch
Tar/gzip	`tar` + `flate2`	Standard combo
Signing verification	`minisign-verify`	Pure Rust, small surface, ed25519 detached sigs
Fuzzy search	`nucleo-matcher`	Used by helix; fast and well-tested
Binary cache format	`postcard`	Compact, fast, schema-stable
MCP (Phase 3)	`rmcp`	SFRS canonical
Errors	`thiserror` (lib), `anyhow` (bin)	Microsoft Rust Guidelines
Logging	`tracing` + `tracing-subscriber`	Per SFRS

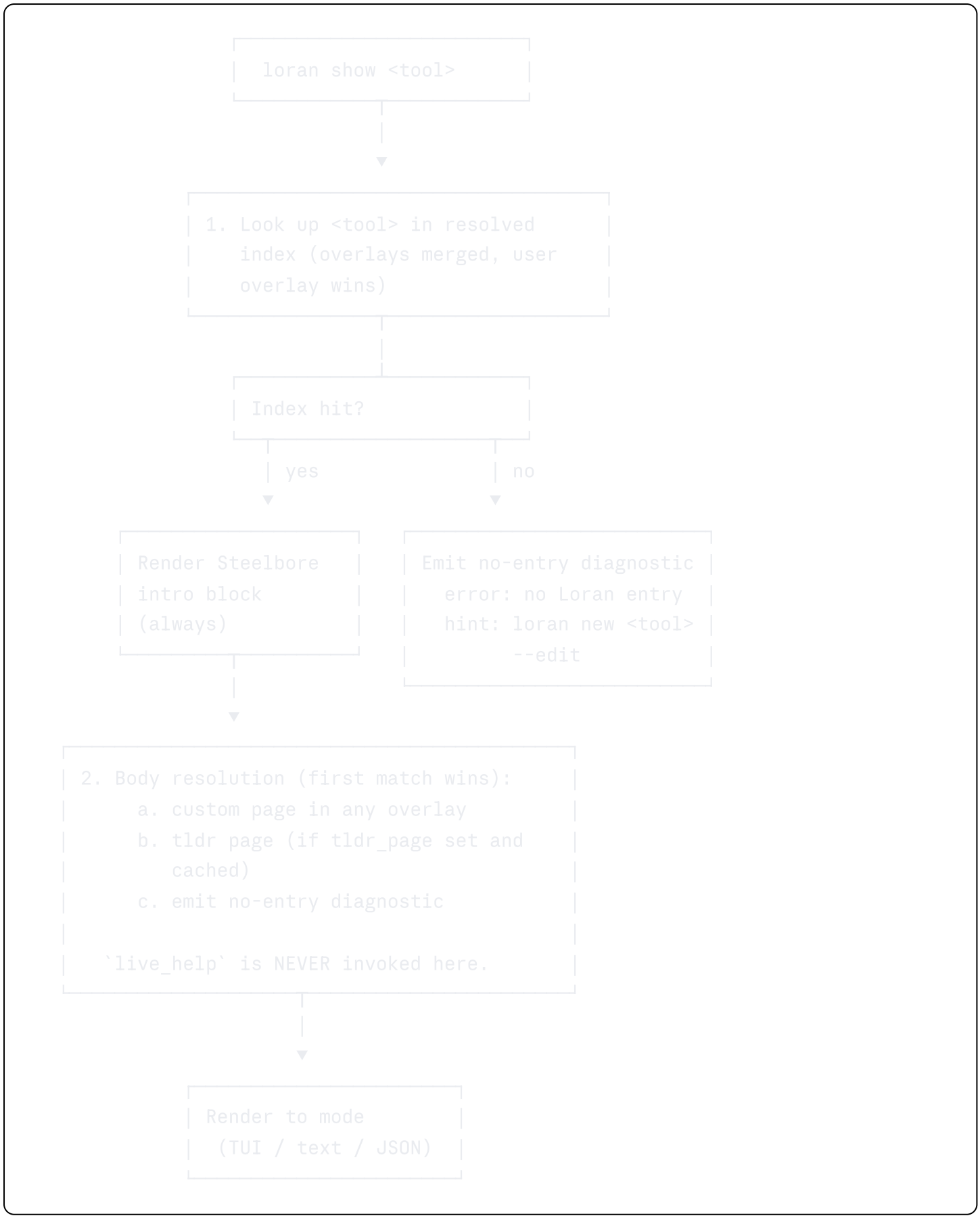
No `tokio` in the v1 fast path. Tarball fetch is one synchronous request; everything else is local I/O. The MCP server may introduce async in Phase 3 — scoped to that crate only.

---

4. Resolution Chains

Two verbs, two distinct flows. The split clarifies brand boundaries: `show` only renders content Steelbore stands behind; `help` is an honest passthrough of upstream tool output.

4.1 `loran show <tool>` — Curated-Or-Fail



`body.kind ∈ {custom, tldr, none}` for the `show` verb.

4.1.1 No-entry response

```
$ loran show widgetctl
error: no Loran entry for 'widgetctl'

  hint: loran new widgetctl --edit
        (scaffolds a page in your user overlay; opens $EDITOR)

  see also: loran search widget --json
            loran help widgetctl  (capture upstream --help directly)
```

In `--json` mode, the structured error carries the same hints (`error.hint` plus `error.see_also`) per SFRS tips-thinking discipline.

4.2 `loran help <tool>` — Always-Live Capture

```
|        uncurated, captured from    |
|        <tool> --help at <ISO 8601  |
|        UTC>"                       |
└────────────────────────────────────┘
```

`body.kind = "live_help"` always for the `help` verb. In `--json`, `data.body.captured_at` carries the ISO 8601 UTC timestamp so agents can cache or invalidate independently.

The de-themed rendering is load-bearing for brand integrity: a user seeing the Steelbore palette should know they are looking at Steelbore-curated content. Live `--help` output is not curated, and its presentation reflects that.

---

5. Filesystem Layout

All paths follow XDG Base Directory Specification.

```
$XDG_DATA_HOME/loran/                       # default: ~/.local/share/loran
├── pages/                                  # upstream Steelbore pages (sync
target)
│   ├── meta.toml                           # tarball version, fetched_at
│   └── <category>/<tool>.md
├── overlays/
│   ├── bravais/<category>/<tool>.md        # distro overlay (read-only at runtime)
│   ├── ferrite/<category>/<tool>.md
│   └── user/<category>/<tool>.md           # user overlay (writable)
└── templates/
    └── tool.md                             # editable scaffold template

$XDG_CACHE_HOME/loran/                      # default: ~/.cache/loran
├── tldr/
│   ├── pages.tar.gz                        # raw tldr tarball
│   ├── extracted/                          # unpacked tree
│   └── meta.toml                           # last-modified, etag
└── index.postcard                          # compiled index for fast startup

$XDG_CONFIG_HOME/loran/                     # default: ~/.config/loran
└── config.toml                             # user preferences (active overlay,
etc.)
```

5.1 Overlay precedence

Lowest to highest:

1. `pages/` — upstream Steelbore pages (synced via tarball)
2. `overlays/<active-distro>/` — distro-specific overrides (Bravais or Ferrite OS)

3. `overlays/user/` — user customisations

Active distro is resolved from `/etc/os-release` at startup (`ID=bravais`, `ID=ferrite`, …). Falls back to "generic" overlay if neither matches. Overridable via `config.toml` (`active_overlay = "bravais"`) and via `--overlay <name>` flag.

Index build merges all three layers; later layers replace earlier ones field-by-field, not record-by-record (so a user can override `summary` without re-stating `category`, `replaces`, etc.).

Overlay source-of-truth: each per-distro overlay is authored and maintained in its own project's repository (the Bravais overlay lives in the Bravais repo, the Ferrite OS overlay in the Ferrite repo), and surfaced into Loran via the upstream tarball publisher pipeline. This keeps distro-specific opinions co-located with the distros that hold them, rather than centralising overlay authorship in the Loran repo.

---

6. Page Format

Single Markdown file with TOML frontmatter, fenced by `+++`.

```markdown
```

````
+++
name           = "eza"
category       = "file-listing"
replaces       = ["ls"]
safe_alias_for = []                  # eza changes ls's default columns and flags;
                                     # `alias ls=eza` is common but breaks some scri
pairs_with     = ["bat", "fd"]
summary        = "Modern ls replacement. Steelbore default for file listing."
official       = "https://eza.rocks"
tldr_page      = "eza"
written_in     = "rust"
since          = "bravais@0.1"
tags           = ["filesystem", "tui-friendly"]
+++

## Steelbore Notes

`eza` is the Steelbore-canonical file lister. Aliased to `ls` in the
default Bravais shell profile (Nushell). Honours `LS_COLORS` and the
Steelbore palette via the `EZA_COLORS` environment variable.

## Recommended Aliases

`````nushell
alias ls = eza --git --icons
alias ll = eza -l --git --icons
alias tree = eza --tree
`````

Differences from `ls`

- Tree mode is built-in (`-T` / `--tree`), no need for separate `tree` binary.
- Git status integration via `--git`.
- ...
````

6.1 Frontmatter schema (required + optional)

Field	Type	Required	Notes
`name`	string	yes	Canonical binary name. Lower-kebab-case.
`category`	string	yes	One of the values in `categories.toml`. May contain `/` as hierarchy separator (v1 renders flat).

Field	Type	Required	Notes
`summary`	string (≤120ch)	yes	One-line description.
`replaces`	array<string>	no	Legacy tool names this supersedes (broad set: modern alternative, not necessarily drop-in).
`safe_alias_for`	array<string>	no	Strict subset of `replaces` flagging tools that can be safely aliased (e.g. `bat` for `cat` in the common case). Validated: `safe_alias_for ⊆ replaces`.
`pairs_with`	array<string>	no	Companion tools that work well alongside this one. Non-reciprocal: A → B does not imply B → A.
`official`	URL	no	Upstream homepage.
`tldr_page`	string	no	tldr key. Defaults to `name` if omitted; set to "" to disable.
`tags`	array<string>	no	Free-form, surfaced in `loran search`.
`written_in`	string	no	Implementation language. Surfaces a "🦀" badge in TUI for `rust`.
`language`	string	RESERVED	Reserved for i18n. When activated in v1.x, translated pages live under `pages.<lang>/` per tldr-pages precedent — not inline.
`since`	string	no	First Steelbore release shipping this tool.
`aliases`	array<string>	no	Alternative spellings (e.g. ripgrep ↔ rg).

Validated by `loran-pages` at index build time. Index build fails loud on schema violations — no silent skipping. The `safe_alias_for ⊆ replaces` invariant is enforced; violations emit `PAGE_PARSE_ERROR` with the offending file + line.

6.2 Categories

Categories are first-class. The category list is a single TOML file shipped in `pages/categories.toml`:

```toml
```

```toml
[file-listing]
title       = "File listing"
description = "Tools that enumerate filesystem entries."

[text-search]
title       = "Text search"
description = "Tools that search file contents."

# ...
```

Slash-tolerance: `[system/file-listing]` is a valid table name today and stored verbatim. The v1 UX renders flat (full path or last segment, configurable). When the catalog grows large enough to justify nested browsing, the data model is already shaped right — only the renderer changes.

User overlays may add categories but cannot remove upstream ones.

---

7. Subcommand Surface

Noun-verb per SFRS §2 Rule 7. Verbs in the canonical set where applicable.

Command	Description
`loran`	TUI if TTY, else `loran list --json`. Auto-detection per SFRS §5.
`loran list`	List tools (filterable). Honours `--category`, `--replaces`, `--safe-alias-for`, `--fields`.
`loran show <tool>`	Show resolved curated page (Steelbore intro + body per §4.1). Curated-or-fail.
`loran help <tool>`	Capture and render `<tool> --help` directly (always-live, de-themed). Per §4.2.
`loran find <legacy>`	Reverse lookup: which tool replaces `<legacy>`? Use `--safe-alias` to filter to alias-safe matches only.
`loran search <query>`	Fuzzy search across name, summary, replaces, tags.
`loran categories`	List categories with counts. JSON-friendly.
`loran new <tool>`	Scaffold a new page from template. `--edit` opens `$EDITOR`.
`loran update`	Refresh upstream `pages/` tarball + tldr tarball. Verifies signatures. Re-builds index.
`loran validate`	Validate all pages against frontmatter schema. CI-friendly.
`loran schema`	JSON Schema (Draft 2020-12) of own data types. SFRS §4.
`loran describe`	Self-description manifest for agents. SFRS §4.
`loran mcp`	Run as read-only MCP server over stdio. Phase 3.

7.1 Global flags (SFRS §3, identical across all Steelbore CLIs)

`--json`, `--format`, `--fields`, `--dry-run`, `--verbose`, `--quiet`, `--no-color`, `--color`, `--help`, `--version`, `--absolute-time`, `--print0`, `--yes`. No deviations.

7.2 `loran new` specifics

Two modes per the prior decision:

- **Interactive (default in TUI / TTY):** prompts for category (with autocomplete from `categories.toml`), summary, replaces; opens `$EDITOR` on the body afterward.
- **Non-interactive:** every field as a flag.

bash

```
# interactive
loran new widgetctl

# non-interactive (scriptable, agentic)
loran new widgetctl \
  --category=file-listing \
  --replaces=ls,dir \
  --safe-alias-for=dir \
  --summary="Widget control utility" \
  --no-edit
```

Writes to `$XDG_DATA_HOME/loran/overlays/user/<category>/<tool>.md` by default. `--scope=upstream` writes into a user-cloned `pages/` checkout (path configured via `config.toml`) for contribution back upstream — the only place git enters the picture, and it's user-side, not bundled.

Template lives at `$XDG_DATA_HOME/loran/templates/tool.md`, populated on first run from a default baked into the binary, then user-editable.

---

8. JSON Envelope

Per SFRS §6. Example for `loran show eza --json`:

json

```
{
  "metadata": {
    "tool": "loran",
    "version": "0.1.0",
    "command": "loran show eza",
    "timestamp": "2026-05-11T08:30:00Z",
    "maintainer": "Mohamed Hammad <Mohamed.Hammad@Steelbore.com>",
    "website": "https://Loran.Steelbore.com/"
  },
  "data": {
    "name": "eza",
    "category": "file-listing",
    "replaces": ["ls"],
    "safe_alias_for": [],
    "pairs_with": ["bat", "fd"],
    "summary": "Modern ls replacement. Steelbore default for file listing.",
    "official": "https://eza.rocks",
    "tags": ["filesystem", "tui-friendly"],
    "written_in": "rust",
    "intro": {
      "source": "steelbore",
      "body_md": "..."
    },
    "body": {
      "kind": "custom",
      "source_path": "/home/mh/.local/share/loran/pages/file-listing/eza.md",
      "body_md": "...",
      "tldr_available": true
    }
  }
}
```


For `loran show`, `body.kind ∈ {custom, tldr, none}`. For `loran help`, the envelope mirrors this but `body.kind = "live_help"` and `data.body.captured_at` carries the spawn timestamp.

---

9. Exit Codes (SFRS §4 + Loran-specific)

Canonical codes 0–5 unchanged. Tool-specific (6–125):

Code	Constant	Meaning
6	`INDEX_NOT_BUILT`	Cache missing; user should run `loran update`.
7	`TARBALL_FETCH_FAILED`	Network or HTTP error during `loran update`.
8	`PAGE_PARSE_ERROR`	Frontmatter schema violation in a discovered page.
9	`LIVE_HELP_TIMEOUT`	`<tool> --help` exceeded 5s timeout under `loran help`.
10	`OVERLAY_WRITE_DENIED`	`loran new` couldn't write to the target overlay.
11	`TARBALL_VERIFY_FAILED`	Minisign signature or SHA-256 mismatch on a fetched tarball. Hard failure; never falls through to extract.

All documented in `loran schema` output.

---

10. TUI Behaviour

- **Default view (no args, TTY):** category list (left pane) + tool list (right pane). Vim `hjkl` navigation; CUA arrow keys. `/` for fuzzy search, `?` for in-app help.
- **Detail view:** Steelbore intro block, then body. Tab-switchable to raw Markdown and frontmatter views (agent-friendly inspection). `pairs_with` entries render as a sidebar.
- `loran help` **capture frame:** monochrome / dim chrome only — NOT the Steelbore palette. Visually distinct from curated content. Honours `NO_COLOR`.
- **Curated content theme:** Steelbore palette only — Void Navy bg, Molten Amber primary text, Steel Blue structural, Radium Green success, Liquid Coolant info, Red Oxide error. Honours `NO_COLOR`.
- **Agent guard rail:** If `AI_AGENT=1` or `AGENT=1` is set, TUI never activates — falls back to `--format json` and warns on stderr per SFRS §5.

---

11. Tarball Update Mechanism

Modelled on tldr-pages, adjusted for Steelbore.

- **Source (upstream pages):** `https://Loran.Steelbore.com/pages/v1/pages.tar.gz`
- **Manifest:** `pages.json` alongside the tarball, contains version + ETag + SHA-256.
- **Signature:** `pages.tar.gz.minisig` alongside the tarball (ed25519 detached signature).
- **Trust root:** the publisher's minisign public key is baked into the binary at compile time (`include_bytes!`). Key rotation requires a new Loran release.

Update flow:

1. **Fetch manifest** via `ureq` + `rustls`, `If-None-Match` against cached ETag. 304 = no work needed.
2. **Fetch tarball + signature** if manifest changed.
3. **Verify SHA-256** against manifest.
4. **Verify minisign signature** against the trust-pinned public key. Hard failure on mismatch → exit code 11 `TARBALL_VERIFY_FAILED`. No extraction attempted on verify failure.
5. **Extract** into `$XDG_DATA_HOME/loran/pages/` atomically (extract to temp dir, rename).
6. **Rebuild index** (postcard cache).

The same flow runs for the tldr tarball (`https://tldr-pages.github.io/assets/tldr.zip`) into the cache dir — but the upstream tldr-pages project does not currently sign its tarballs, so the signature step is skipped for it (SHA-256 against the tldr manifest is the only integrity check available). This asymmetry is documented; a `--require-signatures` flag will refuse the tldr fetch entirely for security-strict deployments.

`--dry-run` reports what would be fetched/extracted/verified without touching disk.

---

12. Agent Surface

12.1 Context files (Day-One artifacts)

Per `steelbore-agentic-cli` §2 and Steelbore Standard v1.1 §5.2:

- `README.md` — includes Project Posture section linking to NOTICE and CONTRIBUTING.
- `NOTICE.md` — full no-warranty / no-liability statement; defers to GPL-3.0-or-later for binding terms.
- `CONTRIBUTING.md` — human onboarding, PR scope, sign-off, security reporting.
- `LICENSE` — verbatim GPL-3.0-or-later text.
- `AGENTS.md` — coding conventions, test commands, repo invariants, forbidden patterns. Generic.
- `CLAUDE.md` — references to skills (`steelbore-standard`, `steelbore-cli-standard`, `steelbore-agentic-cli`, `rust-guidelines`). Claude-specific.
- `SKILL.md` — Loran's own capability surface for the Steelbore Skills system to consume.

All present at repo root from the first commit, before `loran-cli` has any sub-commands.

12.2 MCP Surface (Phase 3, Read-Only)

Loran's full sub-command count (≈13) is borderline for SFRS §2 Rule 8's MCP threshold. We ship MCP because the read-only ones (`list`, `show`, `find`, `search`, `categories`) are unusually high-value for agents discovering what tools exist on a Steelbore system.

The MCP surface is strictly read-only. Agents cannot invoke `update`, `new`, `validate`, or `help` (the live capture). This is a deliberate security and predictability decision:

- `update` would let an agent trigger network I/O and disk writes silently.
- `new` would let an agent write files under the user's overlay.
- `validate` is too verbose for agent token budgets and serves no read-time purpose.
- `help` invokes arbitrary subprocesses; an agent that can choose which `<tool>` to spawn is an attack surface.

The MCP `tools/list` response advertises only the read-only verbs, with names + capability tags. Full schemas come from `tools/get` per `steelbore-agentic-cli` §6 lazy-loading discipline.

12.3 Tips-thinking error catalog

Every error code from §9 has a runnable `hint`. Examples:

Code	Example `hint`
`INDEX_NOT_BUILT`	`loran update`
`NOT_FOUND`	`loran search <query> --json` (with the user's query interpolated)
`PAGE_PARSE_ERROR`	`loran validate --json` (returns the offending file + line)
`OVERLAY_WRITE_DENIED`	`mkdir -p $XDG_DATA_HOME/loran/overlays/user && loran new <tool>`
`LIVE_HELP_TIMEOUT`	`loran new <tool> --edit` (write a curated page instead of relying on --help)
`TARBALL_VERIFY_FAILED`	`loran update --force-refresh` after confirming the publisher key has not rotated; otherwise upgrade Loran.

12.4 Attribution surfacing

Per Steelbore Standard §13.2:

- `--version` (human): footer line `Maintained by Mohamed Hammad <Mohamed.Hammad@Steelbore.com>` and `https://Loran.Steelbore.com/`.
- `--version --json`: `metadata.maintainer` and `metadata.website` fields (see §8).
- `--help`: project URL and maintainer name at footer.
- `README.md`: "Maintainer" section with name, email, project URL.
- TUI About screen: maintainer name, project URL, copyright year.

---

13. Steelbore Standard v1.1 Compliance Checklist

§	Requirement	Status / Note
2	Metallurgical naming	**Deviation acknowledged.** Loran is a heritage engineering acronym (LORAN, radio navigation), not metallurgical. Granted by §5.4 maintainer discretion in recognition of the navigation-as-reference functional metaphor. Release codenames follow the cast-form list (Ingot → Billet → Bloom). Internal crate names follow Rust workspace convention.
3.1	Memory safety	✓ Rust throughout. `rust-guidelines` skill loaded at implementation time.
3.2	Performance / concurrency designed-in	✓ Sync where appropriate; async confined to `loran-mcp` crate. Index build benched.
3.3	Hardened security; PQC	✓ `rustls` for tarball fetch; minisign verification for upstream pages. Minisign is classical ed25519; PQC posture revisited when a stable hybrid signature scheme is available. No crypto subsystem of our own.
4	GPL-3.0-or-later + SPDX	✓ All `.rs` and `Cargo.toml` files. Pages (Markdown) are documents → exempt.
5.2	Required posture files	✓ README.md, NOTICE.md, CONTRIBUTING.md, LICENSE all present at repo root from first commit.
5.1	Default personal-hobby posture	✓ Stated in README Project Posture section.
6.1	POSIX-compliant CLI	✓ SFRS-compliant; default output is POSIX-safe.
7	PFA: no tracking, minimal perms, local	✓ No telemetry. Filesystem + outbound HTTPS to upstream + tldr CDNs only. All data local.
8	CUA + Vim bindings	✓ Both schemes in TUI.
9	Steelbore palette; Void Navy bg	✓ TUI uses palette tokens only for curated content. `loran help` capture frame uses monochrome (intentional brand boundary).
10	FOSS fonts	N/A — terminal app, uses user's terminal font. Docs use Share Tech Mono / Inconsolata per Standard.
11	Material Design / WCAG 2.1 AA	N/A for v1 (no GUI). WCAG-AA contrast already satisfied by palette per §9.

§	Requirement	Status / Note
12	ISO 8601 / UTC / Z-suffix / 24h / metric	✓ All timestamps in JSON envelope and `live_help` captures carry `Z` suffix; no offsets, no local-time in stored or transmitted data.
13	Attribution (maintainer, URL, copyright)	✓ Surfaced in `--version`, `--help`, README, TUI About per §12.4.

---

14. Phasing

Phase	Codename	Scope
Phase 1	Ingot	Workspace + posture files + global flags + JSON envelope + `list` / `show` / `help` / `find` / `search` / `categories` + index from bundled `pages/`. `Ingestor` trait abstraction. Text-mode only.
Phase 2	Billet	Tarball update + minisign signature verification + overlays + TUI + `loran new` + `validate`. The user-visible 1.0 milestone.
Phase 3	Bloom	Read-only MCP surface + `loran schema` JSON-Schema-ified for Anthropic/OpenAI/Gemini/MCP function-calling + `DescribeIngestor` for SFRS describe-compatible Steelbore CLIs + Bravais/Ferrite OS overlay catalogs in tree.

Ingot is shippable on its own as a useful binary, even before tarball/overlay machinery exists. Billet is the user-visible 1.0 milestone. Bloom is the agentic completion.

---

15. Open Questions for v0.3

The following questions from earlier drafts have been resolved and integrated into the relevant sections:

- **Overlay distribution** → each per-distro overlay's source-of-truth lives in its own project repo, surfaced into Loran via the upstream tarball pipeline (documented in §5.1).
- **i18n directory layout** → tldr-pages `pages.<lang>/` directory layout, not inline translations (documented in §6.1).
- **Nested category UX threshold** → ~50 catalog entries is the threshold at which a nested renderer becomes worth implementing; data model is already forward-compatible (documented in §6.2).

Remaining open questions:

1. **Minisign key rotation** — trust-pinned public keys baked into the binary mean rotation requires a new Loran release. Acceptable cadence? Emergency-rotation procedure? Multiple parallel keys for transition windows?
2. `pairs_with` **reciprocity** — current spec treats it as non-reciprocal. Should `loran validate` warn when A claims `pairs_with = ["B"]` but B does not reciprocate? Or accept asymmetry as intentional?
3. `DescribeIngestor` **security model** — Phase 3 wants to spawn `<tool> describe --json` against Steelbore-native binaries. How does Loran decide which binaries are trusted to spawn? Allowlist baked into upstream pages tarball? Self-declaration via SFRS `describe`?

Forged in Steelbore.